Building a Dynamic Website with Django: A Step-by-Step Guide

Samuel Mideksa Follow 5 min read · Feb 25, 2024

26 2

Are you interested in creating your own dynamic website but unsure where to start? Look no further! In this tutorial, we will walk through the process of building a dynamic website using Django, a high-level Python web framework. Even if you have little to no prior programming experience, you can follow along and create your own website.

Introduction

Django is a powerful web framework that simplifies the process of building web applications in Python. It follows the MVC (Model-View-Controller) architecture, which separates the data model, user interface, and application logic. This tutorial will cover the essential steps to create a dynamic website with multiple pages that interact with a database to display data.

Prerequisites

Before we dive into building our website, make sure you have the following prerequisites installed on your system:

- Python

Step 1: Setting Up the Project

1. Create an empty folder for your project.
2. Open the folder in your preferred code editor (e.g., VS Code).
3. Set up a virtual environment to isolate your project dependencies.

```
python -m venv .venv
```

4. Activate the virtual environment.

```
source .venv/bin/activate # For Mac/Linux
.venv\Scripts\activate # For Windows
```

5. Install Django within the virtual environment.

```
pip install django
```

Step 2: Creating the Django Project and App

1. Create a new Django project.

```
django-admin startproject website
```

2. Navigate into the project directory.

```
cd website
```

3. Create a new Django app within the project.

```
python manage.py startapp music
```

4. Add your Django app to the project settings

```
# website/settings.py


INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'music' #add this
]
```

Step 3: Configuring URLs and Views

1. Define views for your web pages inside the app.

```
# music/views.py


from django.shortcuts import render
def home(request):
 return render(request, 'home.html')
def about(request):
 return render(request, 'about.html')
# Define other views (e.g., contact, product) similarly
```

2. Configure URL patterns to map views to URLs.

```
# website/urls.py


from django.contrib import admin
from django.urls import path
from music.views import home, about

urlpatterns = [
 path('admin/', admin.site.urls),
 path('', home),
 path('about/', about),
 # Add URLs for other views here
]
```

Step 4: Creating HTML Templates

1. Create a directory with the name ‘templates’ inside the directory of your app ‘music’ and create HTML templates for each page inside the `templates` directory of your app.

```
<! -- music/templates/home.html -->
```

```html
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Home</title>
</head>
<body>
 <h1>Welcome to My Website</h1>
 <! - Add content here -->
</body>
</html>
```

2. Repeat the above step for other pages like `about.html`, `contact.html`, etc.

Step 5: Setting Up the Database

1. Define models for your database tables inside the app's `models.py` file.

```python
# music/models.py


from django.db import models

class Album(models.Model):
 name = models.CharField(max_length=100)
 artist = models.ForeignKey(Artists) # creates a foreign key to the 'Artists' ta
 release_date = models.DateField()
 # Add more fields as needed
```

2. Run database migrations to create the corresponding tables in the database.

```
python manage.py makemigrations
python manage.py migrate
```

Step 6: Admin Panel Configuration

1. Create a superuser account to access the Django admin panel.

```
python manage.py createsuperuser
```

2. Register your models to manage them via the admin panel.

```
# music/admin.py


from django.contrib import admin
from music.models import Album

admin.site.register(Album)

# registration for other tables in your database
```

3. Set up media files handling if you plan to store files like images in your database.

Get Samuel Mideksa's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

Install pillow module

```
pip install pillow
```

Edit your project settings.

```
# website/settings.py


# Add or edit the following in the static section of the setting (should be at t

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/4.2/howto/static-files/
import os
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles')
STATIC_URL = '/static/'
# Extra places for collectstatic to find static files.
STATICFILES_DIRS = (
    os.path.join(BASE_DIR, 'static'),
)
MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
MEDIA_URL = '/media/'
# Default primary key field type
# https://docs.djangoproject.com/en/4.2/ref/settings/#default-auto-field
DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
```

Edit your project's URLs configuration.

```
# website/urls.py


from django.contrib import admin
```

```
from django.urls import path
from music.views import *
from django.conf import settings
from django.conf.urls.static import static  # Import static setting you just add
urlpatterns = [
    path('admin/', admin.site.urls),
    path('', home),
    path('about/', about),
    # other paths
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)  # Add the par
```

These settings allow Django to serve media files during development.

4. Run the server, access the admin panel at `http://127.0.0.1:8000/admin` and log in with the superuser credentials to add data to your database.

```
python manage.py runserver # to run the project
```

Step 7: Displaying Database Content on Web Pages

1. Modify your views to fetch data from the database and pass it to the templates.

```
# music/views.py


from django.shortcuts import render
from music.models import Album

def home(request):
 Albums = Album.objects.all()
 return render(request, 'home.html', {'Albums': Albums})
```

2. Update your HTML templates to display dynamic content from the database.

```
<! -- myapp/templates/home.html -->
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <title>Home</title>
</head>
<body>
 <h1>Welcome to My Website</h1>
 <ul>
 {% for Album in Albums %}
 <li>{{ Album.name }} - {{ Album.artist }} - {{ Album.release_date }}</li>
 {% endfor %}
 </ul>
</body>
</html>
```

Bonus: Handling Contact Form Submissions

Setting Up the Model

First, let's define a Django model to store user messages. We'll create a model called `User_Messages` with fields for the user's name, email, subject, and message.

```
# music/models.py

from django.db import models
class User_Messages(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    subject = models.CharField(max_length=150)
    message = models.TextField()
```

```
    def __str__(self):
        return self.subject
```

Creating the View

Next, we'll create a view called `contact` to handle the contact form submission. When a user submits the form, we'll retrieve the form data, create a new instance of `User_Messages`, and save it to the database.

```
# music/views.py


from django.shortcuts import render
from .models import User_Messages
def contact(request):
    success = False
    if request.method == 'POST':
        name = request.POST.get('name')
        email = request.POST.get('email')
        subject = request.POST.get('subject')
        message = request.POST.get('message')

        # Create and save a new instance of User_Messages
        user_message = User_Messages(name=name, email=email, subject=subject, me
        user_message.save()
        success = True
    return render(request, 'contact.html', {'success': success})
```

Designing the HTML Template

Now, let's create an HTML template for the contact form. We'll use Bootstrap for styling and include a section to display a success message if the form is submitted successfully.

```
<!-- music/templates/contact.html -->


{% extends 'base.html' %} {% block content %}
<div class="container mt-5">
  <h2>Contact Us</h2>
  <p>Please use the form below to send us your support request or feedback.</p>
  <form action="{% url 'contact' %}" method="POST" class="mt-5">
    {% csrf_token %}
    <!-- Form fields -->
    <button type="submit" class="btn btn-success my-3">Submit</button>
  </form>
</div>
<!-- Success message -->
{% if success %}
<div class="alert alert-success alert-dismissible fade show" role="alert">
  Message successfully sent!
  <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Cl
</div>
{% endif %}
{% endblock %}
```

Conclusion

Congratulations! You have successfully created a dynamic website using Django. By following this tutorial, you've learned the fundamental steps involved in building a web application, including setting up the project, defining URLs and views, creating HTML templates, configuring the database, and displaying dynamic content.

Feel free to explore Django further and customize your website according to your preferences and requirements.

To access the full code repository and test the website yourself, you can visit the GitHub repository here

Happy coding! 🚀

Web Development | Python | Django | Programming | Computer Science

Written by Samuel Mideksa

18 followers · 1 following

Follow

Responses (2)

Write a response

What are your thoughts?

Ewho Ruth
Nov 18, 2024

very practical article and it simple and clean

Reply

Dr Raiprasad
Aug 1, 2024

I want to know if the codes written in article for website building will also be valid for higher versions of Django.

I use Django5.12

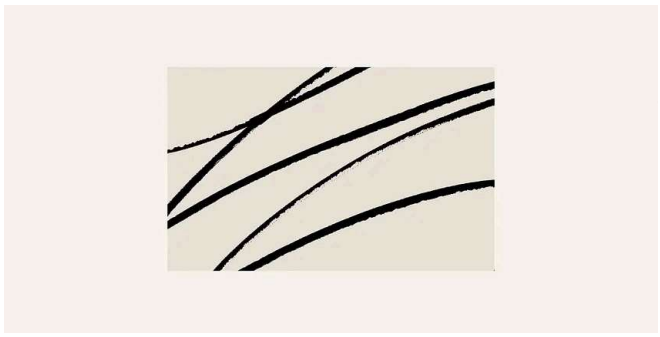
Reply

More from Samuel Mideksa

Samuel Mideksa

JavaScript API: Exploring Movie Data

In this guide, we'll walk you through the



Samuel Mideksa

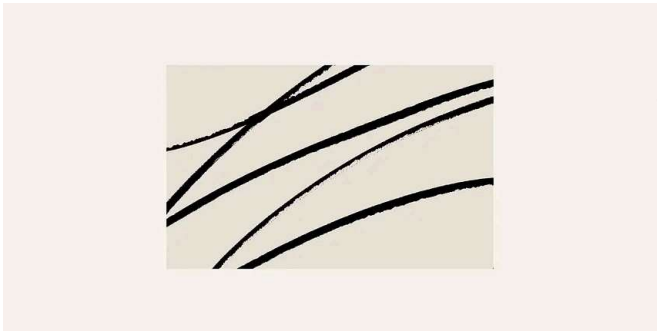
Web Scraping using Python

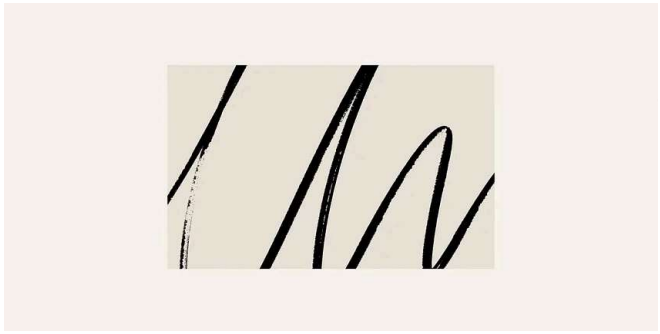
Web scraping and sending to telegram channel using a telegram bot, using python.

Open in app ↗

Sign up Sign in

Medium Search Write





Samuel Mideksa

Building My Portfolio Website

How to build a portfolio website and deploy it for free using only GitHub.

Nov 25, 2023 3

Samuel Mideksa

Cloning a Website Using Bootstrap

Introduction

Dec 2, 2023

See all from Samuel Mideksa

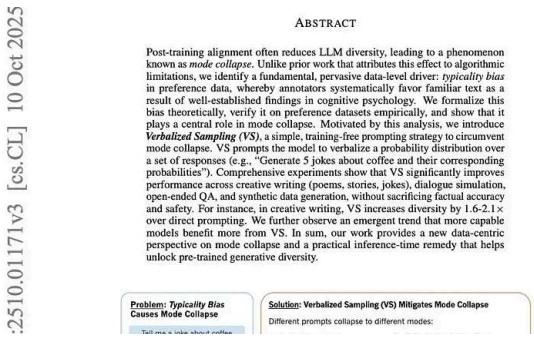
Recommended from Medium

TechByRahmat

20 GitHub Full-Stack Projects You Can Learn From

Real-world codebases that teach you more than any course ever will.

Oct 14, 2025 54

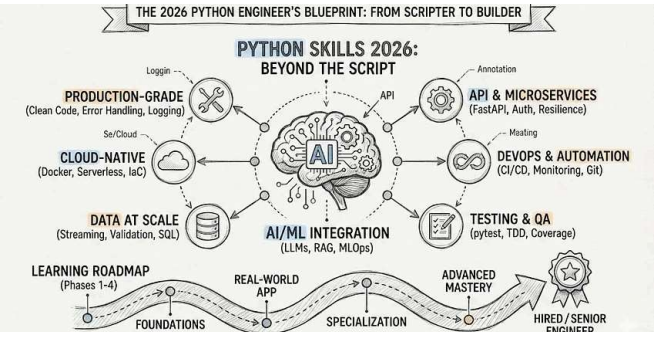


In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

Oct 20, 2025 24K 634

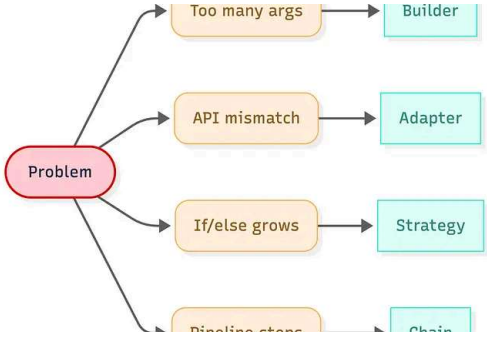


In Python in Plain English by Faisal haque

The Python Skills That Actually Get You Hired in 2026 (And How to...

Why “Knowing Python” Isn’t Enough Anymore

Feb 8 602 22

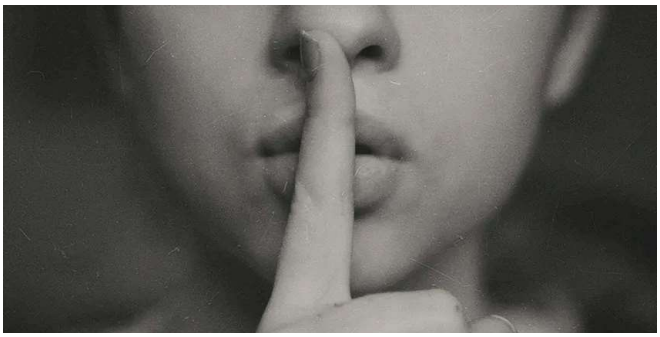


In Women in Technology by Alina Kovtun

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

Jan 29 4.2K 32



Anas Issath

Environment Variables: The First Thing Every Backend Engineer...

Your secrets are probably in your Git history right now. Let’s fix that before it costs you...

Jan 25 83

Yogeshkrishnanseeniraj

Building AI Agents in Django SaaS: LangChain + OpenAI for...

The rise of AI agents has transformed how we build SaaS applications. Unlike traditional...

Jan 5 1

See more recommendations